Introduction to OOPS Programming

Total Time: 3 Hours

Section A: Concept Application (Theoretical – Scenario Based)

1. You are building a **food delivery application**. At which stage would you prefer using a **compiler** over an **interpreter**, and why? Explain based on execution speed, error detection and deployment.

Ans : In a food delivery application, I would prefer using a compiler during the final development and deployment stage.

1. Execution Speed

- A compiler translates the entire program into machine code before execution.
- The compiled program runs much faster because the code does not need to be translated line by line each time it runs.
- For a food delivery app handling many users, orders, and payments, fast execution improves performance and user experience.

2. Error Detection

- A compiler checks the whole program before creating the executable file.
- Many syntax and type errors are detected at once, making the application more reliable.
- This helps developers fix issues before the app is released to customers.

3. Deployment

- Compiled programs can be distributed as executable files without requiring the source code.
- Users and servers can run the application directly, which improves security and efficiency.
- Deployment is easier because the target system only needs the executable, not the compiler

2. **You are asked to design a program that calculates employee salary with multiple conditions (bonus, tax, overtime). Which programming constructs would you combine to solve this efficiently?**

Ans : To calculate an employee's salary with conditions such as bonus, tax, and overtime, I would combine the following programming constructs:

1. Variables – Store salary, bonus, overtime pay, tax, and final salary.
2. Input/Output Statements – Take employee details and display the final salary.
3. Conditional Statements (if-else) – Apply bonus, tax, or overtime rules based on specific conditions.
4. Arithmetic Operators – Perform salary calculations.
5. Functions – Separate tasks like calculating bonus, tax, and overtime to make the program organized and reusable.
6. Loops (if multiple employees are processed) – Calculate salaries for several employees efficiently.

3. **While developing a menu-driven console application, why are loops preferred over conditional repetition? Explain which loop you would choose and why?**

Ans :

In a menu-driven console application, loops are preferred over conditional repetition because the menu needs to be displayed repeatedly until the user chooses to exit. Using loops avoids writing the same code multiple times and makes the program easier to maintain.

Why use loops?

- Repeats the menu automatically.
- Reduces code duplication.
- Makes the program more efficient and organized.
- Allows the user to perform multiple operations without restarting the program.

Which loop would I choose?

I would choose a do-while loop because the menu should be displayed at least once before checking whether the user wants to continue or exit.

4. **A student writes a program to calculate average marks using int variables and gets incorrect output. Identify the probable cause and explain how choosing the correct data type solves the issue.**

Ans :

When the student uses `int` variables, the result of the division is also an integer, so any decimal part is discarded.

Example

```
int marks1 = 85, marks2 = 90, marks3 = 88;
int average;

average = (marks1 + marks2 + marks3) / 3;
printf("%d", average);
```

Output:

87

The actual average is:

(85 + 90 + 88) / 3 = 87.67

But since `average` is an `int`, the decimal part (`.67`) is lost.

Solution

Use a floating-point data type such as `float` or `double`.

```
float average;
average = (marks1 + marks2 + marks3) / 3.0;
printf("%.2f", average);
```

Output:

87.67

The incorrect output occurs because `int` variables store only whole numbers and perform integer division. Using `float` or `double` allows the program to store decimal values and calculate the average accurately.

5. **You are debugging a program that complies successfully but crashes at runtime. Which category of error is this? Mention two debugging practices you would follow to resolve it.**

Ans :

This is a runtime error because the program compiles successfully but crashes while it is executing.

Two debugging practices to resolve it:

1. Use print statements or a debugger
   - Print variable values at different points in the program or use a debugger to find where the crash occurs.
   - This helps identify incorrect values or problematic code sections.
2. Check for common runtime issues
   - Verify array indices are within bounds.
   - Ensure pointers are initialized before use.
   - Check for division by zero and invalid memory access.
   - These are common causes of runtime crashes.

6. **You need to store student records with name, marks, and grade. Why would you choose a structure over multiple arrays in this scenario?**

Ans :

I would choose a structure because it groups all related information about a student into a single unit.

Advantages of using a structure

1. Better Organization
   - A structure stores `name`, `marks`, and `grade` together for each student.
   - This makes the data easier to understand and manage.
2. Simpler Code
   - Instead of maintaining separate arrays for names, marks, and grades, one structure can hold all the information.
   - This reduces confusion and programming errors.
3. Easy Access
   - Student details can be accessed using a single variable.
   - Example:

```
 student.name
student.marks
student.grade
```

4. Improved Maintainability
    - If a new field (such as age or roll number) needs to be added later, it can be added to the structure without creating additional arrays.

Section B: Practical Coding Tasks

1. **Conditional Logic Implementation**
   Design a **"Study Mood Assistant"** that:
   - Takes user input: hours studied today
   - Displays motivation messages based on conditions
   - Uses **if-else ladder or switch case**

2. **Loops + Arrays**
   Create a program that:
   - Accepts 7 **days of screen time data**
   - Calculates and prints:
     - ➔ Total screen time
     - ➔ Average screen time
   - Displays a warning if average exceeds healthy limit

3. **Functions & Pointers**
   Write a program to:
   - Swap two numbers **using a user-defined function**
   - Implement swapping using **pointers**
   - Explain why pass-by-reference is necessary here

4. **File handling**
   Build a program that:
   - Writes daily goals to a file
   - Reads and displays them on program restart
   - Uses correct file modes

Section C: Mini Capstone Project

Mini Project: Student Productivity Tracker

Objective

Build a **console-based Student Productivity Tracker** that logs daily study hours and generates a weekly report.

Your application must:

1. Menu-driven program
2. Log daily study hours
3. Generate weekly report
4. Store data using files